

گزارش فنی پروژه مدیریت بیمارستان

مقدمه و تحلیل نیازمندی‌ها

۱-۱. مقدمه

با افزایش حجم اطلاعات و فعالیت‌های روزمره در مراکز درمانی، مدیریت اطلاعات بیماران، پزشکان، بخش‌ها، نوبت‌ها و هزینه‌ها به صورت دستی می‌تواند باعث افزایش خطا و دشواری در دسترسی به اطلاعات شود.

هدف از این پروژه، طراحی و پیاده‌سازی یک سیستم مدیریت بیمارستان با زبان برنامه‌نویسی Java است که بتواند بخش‌های اصلی مدیریت یک بیمارستان را به صورت یکپارچه مدیریت کند.

این برنامه به صورت یک نرم‌افزار دسکتاپ طراحی شده و رابط کاربری آن با استفاده از **Java Swing** پیاده‌سازی شده است. همچنین برای ذخیره‌سازی دائمی اطلاعات از **SQLite** و **JDBC** استفاده شده است.

ساختار پروژه نیز به صورت تفکیک‌شده طراحی شده است تا بخش رابط کاربری، منطق اصلی برنامه و دسترسی به داده‌ها از یکدیگر جدا باشند.

پروژه به صورت کامل در گیت هاب به لینک زیر قابل دسترسی است:

<https://github.com/sh4harvard/Hospital-Management-System/tree/master>

۲. تحلیل نیازمندی‌های پروژه

سیستم طراحی شده باید امکانات اصلی زیر را فراهم کند:

❖ مدیریت بیماران

سیستم باید امکان:

- اضافه کردن بیمار
- مشاهده اطلاعات بیمار
- ویرایش اطلاعات بیمار
- حذف بیمار
- اختصاص بیمار به بخش
- انتقال بیمار از یک بخش به بخش دیگر
- ترخیص بیمار
- مشاهده سوابق نوبت‌های بیمار
- مشاهده پرونده پزشکی
- مشاهده صورت حساب بیمار

را فراهم کند.

هر بیمار دارای اطلاعاتی مانند شناسه، نام، سن، جنسیت و شماره تلفن است.

علاوه بر این، هر بیمار دارای یک MedicalRecord یا سابقه پزشکی، یک Bill صورت حساب، و مجموعه‌ای از Appointment قرار ویزیت ها است.

❖ مدیریت پزشکان

سیستم امکان مدیریت پزشکان را نیز فراهم می‌کند.

برای هر پزشک اطلاعاتی مانند:

- شناسه
- نام
- سن
- جنسیت
- شماره تلفن
- تخصص
- بخش مربوطه
- ساعات کاری
- ظرفیت روزانه نوبت‌ها

نگهداری می‌شود.

امکانات مربوط به پزشکان شامل:

- اضافه کردن پزشک
- مشاهده اطلاعات پزشک
- ویرایش پزشک
- حذف پزشک
- اختصاص پزشک به بخش
- انتقال پزشک بین بخش‌ها
- مشاهده نوبت‌های پزشک

نیز وجود دارد.

❖ مدیریت بخش‌ها

بیمارستان می‌تواند دارای چند بخش مختلف باشد.

هر Ward (بخش) دارای:

- شناسه
- نام
- ظرفیت
- لیست بیماران
- لیست پزشکان

است.

سیستم هنگام پذیرش بیمار ظرفیت بخش را بررسی می‌کند و از ورود بیمار به بخش پر جلوگیری می‌کند.

همچنین امکان انتقال بیمار بین بخش‌ها نیز وجود دارد.

❖ مدیریت نوبت‌ها

یکی از بخش‌های اصلی سیستم، مدیریت Appointment است.

هنگام ایجاد یک نوبت، سیستم موارد مختلفی را بررسی می‌کند؛ از جمله:

- بیمار معتبر باشد.
- پزشک معتبر باشد.
- پزشک در تاریخ موردنظر ظرفیت داشته باشد.
- زمان انتخاب‌شده در شیفت کاری پزشک قرار داشته باشد.
- پزشک در همان زمان نوبت دیگری نداشته باشد.
- در صورت بستری بودن بیمار، پزشک مربوط به همان بخش باشد.

نوبت‌ها همچنین دارای وضعیت هستند و امکان تکمیل به معنای اتمام یا لغو آنها وجود دارد.

۳. پرونده پزشکی و صورت حساب

برای هر بیمار یک MedicalRecord ایجاد می‌شود.

این بخش برای نگهداری اطلاعات پزشکی بیمار طراحی شده است و به شیء Patient متصل است.

همچنین هر بیمار دارای یک Bill یا صورت حساب است.

در صورت استفاده بیمار از خدمات پزشکی، یک Charge برای آن خدمت ایجاد شده و به صورت حساب بیمار اضافه می‌شود.

بنابراین هزینه‌های بیمار به صورت جداگانه ذخیره شده و امکان محاسبه مجموع هزینه‌ها وجود دارد.

۴. خدمات پزشکی

سیستم دارای کلاس MedicalService است.

هر خدمت پزشکی دارای اطلاعات مربوط به خود بوده و می‌تواند در صورت استفاده برای بیمار، به صورت یک هزینه (Charge) در صورت حساب بیمار ثبت شود.

همچنین سیستم درآمد حاصل از خدمات پزشکی را در بخش درآمدهای بیمارستان ثبت می‌کند.

۵. مدیریت درآمد بیمارستان

برای مدیریت درآمد بیمارستان، یک interface با نام:

HospitalIncome

در نظر گرفته شده است.

انواع مختلف درآمد می‌توانند این interface را پیاده‌سازی کنند و زیر نظر آن هستند.

از جمله:

- درآمد حاصل از خدمات پزشکی
- تشویقی مربوط به خالی شدن کامل بیماران یک بخش

برای درآمد مربوط به بخش، کلاس WardBonus ایجاد شده است.

طبق منطق پروژه، زمانی که آخرین بیمار یک بخش ترخیص شود و بخش خالی شود، درآمد مربوط به خالی شدن آن بخش ثبت می‌شود.

این درآمد در مجموع بودجه بیمارستان نیز محاسبه می‌شود.

۶. طراحی شی گرا

یکی از اهداف اصلی پروژه استفاده از مفاهیم برنامه‌نویسی شی گرا در طراحی سیستم بوده است.

کلاس Person

کلاس Person به عنوان کلاس پایه برای موجودیت‌های انسانی سیستم طراحی شده است.

کلاس‌های:

- Patient
- Doctor

از آن ارث‌بری می‌کنند.

اطلاعات مشترک مانند:

- ID
- Name
- Age
- Gender
- Phone Number

در این کلاس قرار گرفته‌اند.

وراثت

از inheritance برای جلوگیری از تکرار اطلاعات مشترک بین Patient و Doctor استفاده شده است. به این ترتیب ویژگی‌های مشترک در Person قرار گرفته و هر کلاس فرزند ویژگی‌های مخصوص خود را دارد.

Abstract Class

کلاس Person به صورت abstract طراحی شده تا خود به تنهایی قابلیت وجود نداشته و فقط زیرمجموعه‌های آن تشکیل شود و متد showInfo() را به عنوان متدی برای نمایش اطلاعات تعریف می‌کند. هر کلاس فرزند پیاده‌سازی مخصوص خود را برای نمایش اطلاعات دارد.

Interface

در پروژه از interface نیز استفاده شده است.

برای مثال:

DisplayInformation

برای کلاس‌هایی استفاده شده که قابلیت نمایش اطلاعات خود را دارند.

همچنین HospitalIncome برای مدل سازی انواع درآمدهای بیمارستان استفاده شده است.

۷. کلاس اصلی HospitalSystem

کلاس HospitalSystem بخش اصلی منطق سیستم را مدیریت می کند.

این کلاس مجموعه های اصلی سیستم را نگهداری می کند:

- بیماران
- پزشکان
- بخش ها
- نوبت ها
- خدمات پزشکی
- درآمدهای بیمارستان

همچنین عملیات اصلی سیستم مانند:

- اضافه کردن
- حذف کردن
- پیدا کردن
- پذیرش
- ترخیص
- انتقال
- ایجاد نوبت
- ایجاد هزینه
- محاسبه درآمد

در این کلاس مدیریت می شوند.

به این ترتیب منطق اصلی برنامه از رابط گرافیکی جدا باقی می ماند.

۸. رابط گرافیکی

رابط گرافیکی پروژه با استفاده از **Java Swing** ساخته شده است.

ساختار رابط کاربری از چندین `JPanel` تشکیل شده است.

صفحه اصلی برنامه شامل بخش هایی مانند:

- Hospital
- Patients
- Doctors
- Appointments

است.

برای جابه جایی بین صفحات مختلف از `CardLayout` استفاده شده است.

به عنوان مثال، `ContentPane` صفحات مختلف را نگهداری می کند و با استفاده از متدهایی مانند:

- `showPatients()`
- `showDoctors()`
- `showAppointments()`
- `showHospital()`

صفحه موردنظر را نمایش می دهد.

۹. صفحات مربوط به بیماران

برای مدیریت بیماران چند پنل جداگانه طراحی شده است.

PatientPanel

لیست بیماران را در قالب جدول نمایش می دهد.

PatientAddPanel

برای اضافه کردن بیمار جدید استفاده می شود.

PatientEditPanel

برای ویرایش اطلاعات بیمار استفاده می شود.

PatientDetailPanel

اطلاعات کامل بیمار را نمایش می دهد.

این جداسازی باعث شده عملیات مختلف مربوط به بیماران در پنل های جداگانه قرار بگیرند و رابط کاربری ساده تر باشد.

۱۰. صفحات مربوط به پزشکان

ساختار مشابهی برای پزشکان ایجاد شده است.

پنل های مربوط به پزشک شامل:

- DoctorPanel
- DoctorAddPanel
- DoctorEditPanel
- DoctorDetailPanel

هستند.

این پنل‌ها وظایف مربوط به مشاهده، ایجاد و ویرایش پزشکان را مدیریت می‌کنند.

۱۱. مدیریت بخش‌ها

برای مدیریت بخش‌های بیمارستان نیز چند پنل طراحی شده است.

WardPanel

لیست بخش‌ها، ظرفیت و تعداد بیماران و پزشکان هر بخش را نمایش می‌دهد.

WardDetailPanel

اطلاعات یک بخش مشخص و بیماران و پزشکان موجود در آن را نمایش می‌دهد.

همچنین امکان ترخیص بیمار از بخش از طریق این صفحه وجود دارد.

۱۲. مدیریت پایگاه داده

برای ذخیره دائمی اطلاعات از **SQLite** استفاده شده است.

دسترسی به پایگاه داده در **package** جداگانه **Database** قرار گرفته است. ساختار فعلی مخزن نیز این جداسازی را نشان می‌دهد؛ در کنار **package** مربوط به منطق **Hospital**، **package** جداگانه **Database** وجود دارد.

کلاس‌های موجود در این بخش وظایفی مانند:

- اتصال به پایگاه داده
- ایجاد ساختار پایگاه داده
- خواندن اطلاعات
- ذخیره اطلاعات

را انجام می‌دهند.

این جداسازی باعث می‌شود منطق مربوط به SQLite مستقیماً در کلاس‌های رابط کاربری یا مدل‌های اصلی برنامه قرار نگیرد.

۱۳. ذخیره اطلاعات هنگام خروج

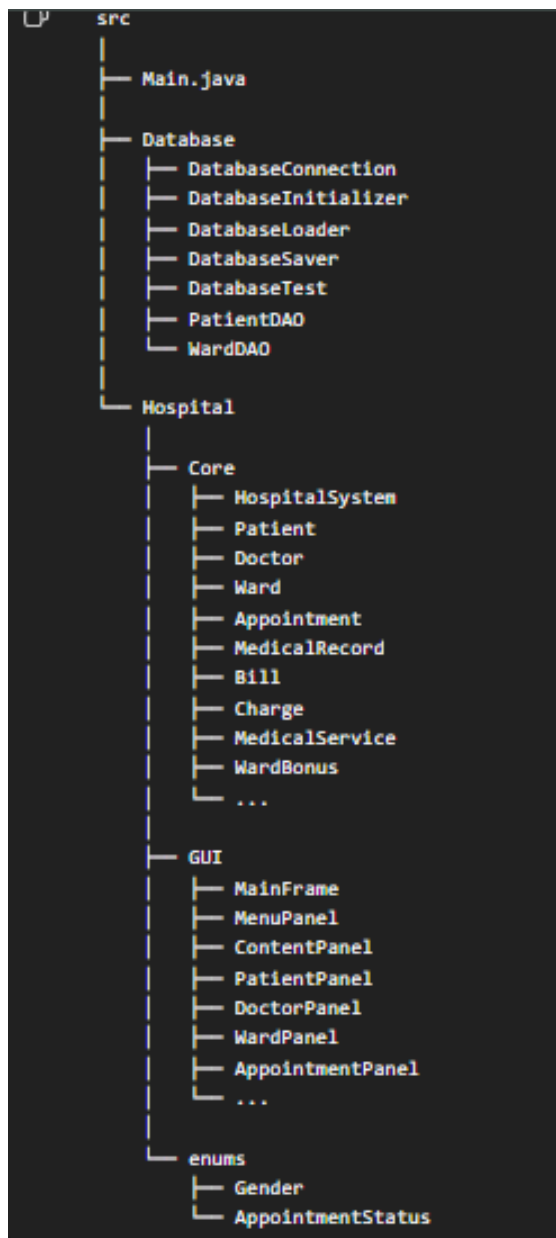
برنامه هنگام بسته شدن پنجره اصلی، اطلاعات موجود در سیستم را در پایگاه داده ذخیره می‌کند.

این کار در `MainFrame` و با استفاده از `WindowListener` انجام شده است.

در نتیجه اطلاعاتی که در طول اجرای برنامه تغییر کرده‌اند، قبل از بسته شدن برنامه ذخیره می‌شوند و پایگاه داده با آخرین اطلاعات بروز می‌شود.

۱۴. ساختار پروژه

ساختار اصلی پروژه به صورت کلی به شکل زیر است: (عکس برگرفته شده از Chat-GPT)



در repository فعلی نیز package های Database و Hospital در زیر src قرار گرفته‌اند.

این ساختار باعث شده پروژه از نظر مسئولیت‌ها به بخش‌های مشخص تقسیم شود.

۱۵. نمودارهای مورد نیاز پروژه

بر اساس آیین نامه، سه نوع نمودار اصلی باید در گزارش قرار داده شوند:

۱۵-۱. Use Case Diagram

بازیگران اصلی سیستم می توانند شامل:

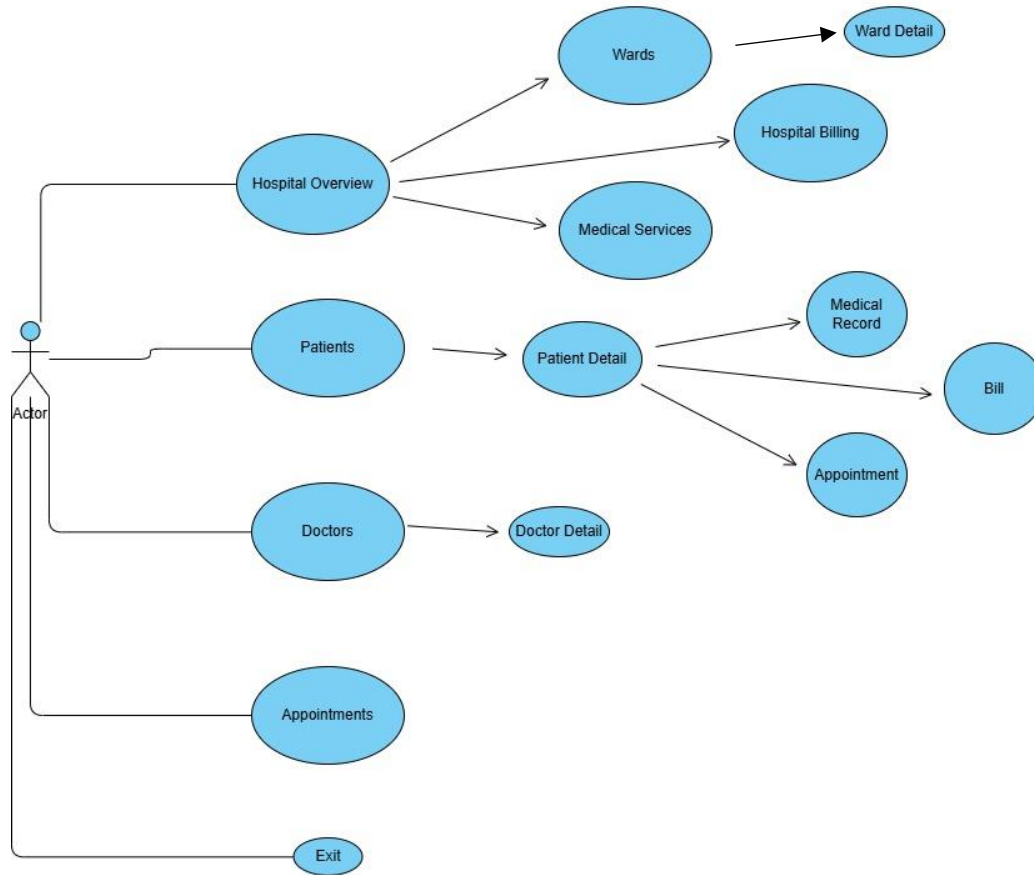
Administrator / Hospital Staff

باشند.

Use Case های اصلی:

- مدیریت بیمار
- مدیریت پزشک
- مدیریت بخش
- مدیریت نوبت
- مدیریت خدمات پزشکی
- مدیریت صورت حساب
- مشاهده اطلاعات بیمارستان
- مدیریت درآمد

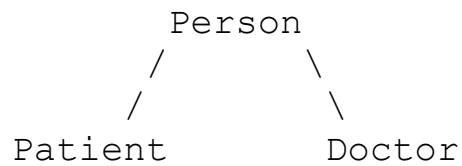
Hospital Management System



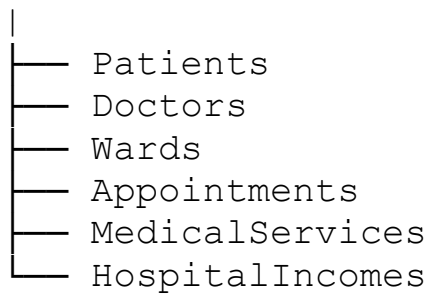
۲-۱۵. Class Diagram

نمودار کلاس پروژه باید ارتباط بین کلاس‌های اصلی را نشان دهد.

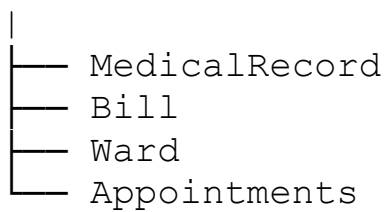
مهم‌ترین ارتباط‌ها:



HospitalSystem



Patient

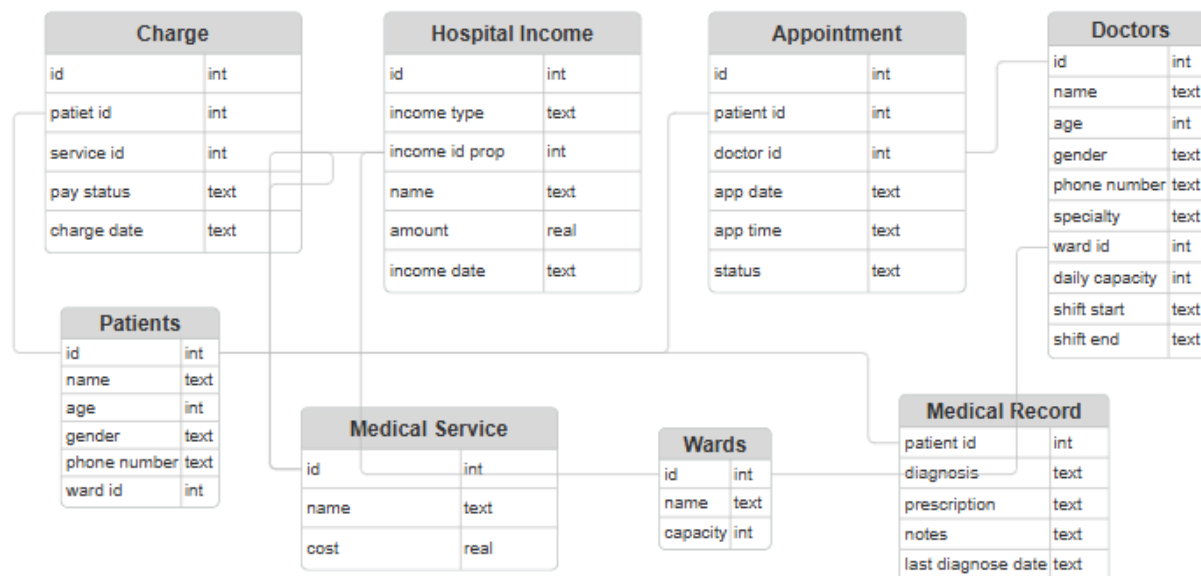


Ward



Patient — Appointment — Doctor

ER Diagram



۱۶. Sequence Diagram

برای Sequence Diagram سناریوی پذیرش بیمار در بخش انتخاب میکنم، زیرا چندین کلاس اصلی پروژه را درگیر می‌کند.

روند کلی:

```

sequenceDiagram
    participant User
    participant Patient GUI
    participant HospitalSystem

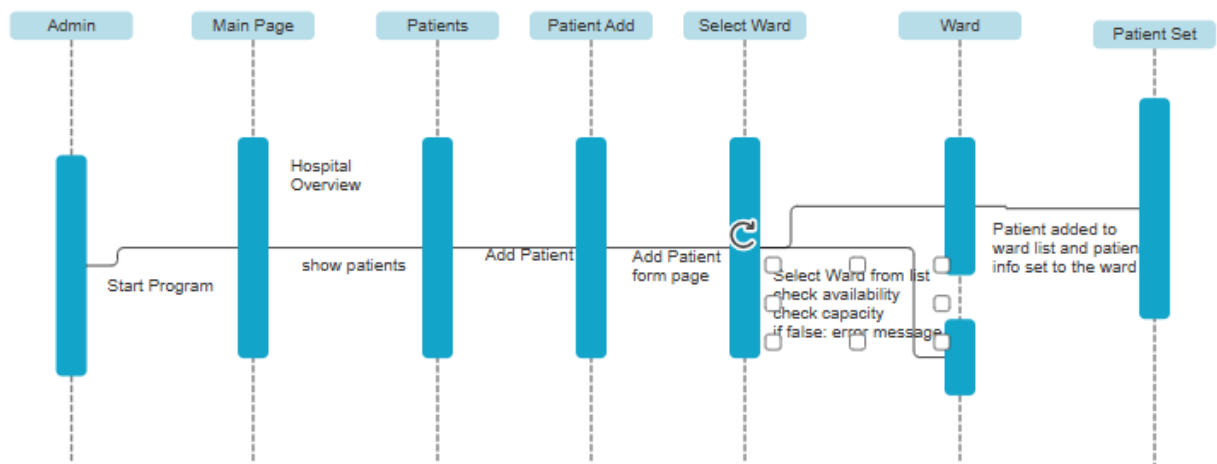
    User->>Patient GUI: Select Patient
    Patient GUI->>HospitalSystem: admitPatient(patient, ward)
    
```

```

| addPatient(patient)
↓
Ward
|
| Check Capacity
|
| Add Patient
↓
Patient
|
| setWard(ward)
↓
Ward

```

ابتدای اجرای برنامه که وارد صفحه اصلی بیمارستان می شویم، سپس در بخش مدیریت بیمار ها. از دکمه اضافه کردن بیمار استفاده و بعد فرم اطلاعات بیمار نمایش داده میشود و در کنار اطلاعات شخصی و اولیه بیمار گزینه انتخاب بخش برای بستری نیز به صورت لیست بخش ها وجود دارد. که پس از انتخاب بخش و تایید مجاز بودن پذیرش (شامل چک کردن معتبر بودن و ظرفیت بخش مورد نظر) آن بیمار به بخش اضافه شده و در لیست بیماران در جدول در قسمت بخش اعمال و همچنین در لیست بیماران آن بخش نیز اضافه میود. در صورت پر بودن بخش، عملیات پذیرش با مقدار false برمی گردد و رابط کاربری پیام مناسب را نمایش می دهد.



۱۷. چالش‌های پیاده‌سازی

در طول پیاده‌سازی پروژه، مهمترین چالش‌های اصلی مدیریت ارتباط بین موجودیت‌های مختلف، و بررسی حالت‌های مختلف در آن شرایط بود. برای مثال، بیمار هم به بخش، هم به نوبت‌ها، هم به پرونده پزشکی و هم به صورت حساب خود ارتباط دارد. و در هنگام حذف بیمار و یا تغییر اطلاعات آن باید تمامی اطلاعات صفحه‌های دیگر نیز به درستی اعمال میشد و تمامی جداول اطلاعاتی بروز شده و همچنین از ذخیره شدن آن اطلاعات در دیتابیس نیز اطمینان حاصل میشد.

همچنین هنگام حذف یا انتقال یک موجودیت باید ارتباط‌های مربوط به آن نیز به درستی به‌روزرسانی شوند. به عنوان مثال هنگام حذف یک بخش از بیمارستان تمامی اطلاعات مربوط به آن نیز باید بروز میشد، علاوه بر چک کردن شرایط حذف بخش (از جمله خالی بودن از بیمار و پزشک)، باید در تمامی اطلاعات و فرم‌ها و جداول این تغییرات اعمال می‌شد و همچنین به دلیل حذف کلی بخش هرگونه تشویقی‌هایی که مربوط به آن بود و در لیست درآمد بیمارستان اعمال شده بود نیز حذف میشد.

یکی دیگر از چالش‌ها، هماهنگ نگه داشتن اطلاعات حافظه و پایگاه داده بود.

در بخش رابط کاربری نیز مدیریت تعداد زیادی صفحه یکی از چالش‌ها بود که با استفاده از CardLayout و یک ContentPanel مرکزی و ایجاد یک پنل مرکزی و ارسال به تمامی پنل‌ها به عنوان ورودی برای جلوگیری از ایجاد پنل مرکزی‌های متعدد در صورت جابجایی بین بخش حل شد.

۱۸. نتیجه‌گیری

در این پروژه یک سیستم مدیریت بیمارستان با استفاده از زبان Java طراحی و پیاده‌سازی شد.

سیستم قابلیت مدیریت موجودیت‌های اصلی بیمارستان شامل بیماران، پزشکان، بخش‌ها، نوبت‌ها، خدمات پزشکی و اطلاعات مالی را دارد.

در طراحی پروژه از مفاهیم اصلی برنامه‌نویسی شی‌گرا مانند:

- Encapsulation
- Inheritance
- Abstraction
- Polymorphism
- Interface

استفاده شده است.

همچنین رابط گرافیکی با Java Swing و ذخیره‌سازی دائمی اطلاعات با SQLite پیاده‌سازی شده است.

یکی از اهداف مهم پروژه، جدا کردن منطق اصلی سیستم از رابط کاربری و بخش پایگاه داده بوده است تا ساختار برنامه قابل فهم‌تر و توسعه آن در آینده ساده‌تر باشد.

و همچنین برای رابط کاربری GUI و طرح دیزاین آن از یک design sheet واحد استفاده و در تمامی پنل‌ها از آن به کار برده شد تا در صورت علاقه به تغییر آن به راحتی و از دیزاین شیت مرکزی قابل ادیت بود.

در نهایت، این پروژه علاوه بر پیاده‌سازی یک سیستم کاربردی مدیریت بیمارستان، باعث تمرین و استفاده عملی از مفاهیم برنامه‌نویسی شی‌گرا، طراحی GUI، مدیریت رویدادها، کار با پایگاه داده و سازمان‌دهی یک پروژه نسبتاً بزرگ Java شده است.

همچنین این را هم اعلام کنم که علاقه بسیار زیادی داشتم که بخش‌های امتیازی را هم بتوانم انجام بدم اما بدلیل کمبود زمان قادر نشدم.

۱۹. استفاده از هوش مصنوعی

در روند انجام این پروژه از ابزارهای هوش مصنوعی به عنوان ابزار کمک‌آموزشی استفاده شده است. حتی در پرامپ آن هم اعلام کردم که همانند دوره قبلی که گذراندم (من دوره آموزش مبانی ورود به علوم کامپیوتر و هوش مصنوعی هم به همین صورت از هوش مصنوعی استفاده کردم) کد و برنامه به من تحویل نده، من دانشجوی

کارشناسی رشته کامپیوتر هستم و هدفم از این پروژه یادگیری و پیشرفت هست. برای این پروژه از دو هوش مصنوعی Chat-GPT و CLAUDE استفاده کردم. استفاده از این ابزار برای توضیح و یادگیری مفاهیم برنامه‌نویسی Java و به خصوص برای یادگیری Java Swing، مشورت و بحث برای طراحی ساختار کاربردی و زیبا، بررسی خطاها و دیباگینگ، پیشنهاد راه‌حل برای مشکلات پیاده‌سازی و کمک به درک ساختار برخی قسمت‌های پروژه انجام شده است. کدهای پروژه توسط خودم بررسی، اجرا، آزمایش و در موارد مختلف اصلاح شده‌اند. از هوش مصنوعی برای تهیه مستقیم و بدون بررسی کد و کل پروژه استفاده نشده است.

با تشکر از زحمات، وقت گذاشتن و ارزش قائل شدن شما

شایان منفرد

2026